

Approximation Algorithms

Motivation

- NP Complete problems take too long.
vertex cover, travelling salesman problem, etc.
- Approximation algorithms provide near-optimal solution with a guarantee
 - Approximation ratio (how far from optimal answer)
- Approx. algo take polynomial time

Polynomial-time Approximation Scheme (PTAS)

- Better approximation at the cost of more computation time.
 - tradeoff between compute time vs quality of approximation

Definition: Approximation scheme for an optimization problem is an approximation algorithm that takes an input instance of the problem and a fixed value of ϵ ($\epsilon > 0$), and provides $(1+\epsilon)$ -approximation ratio.

- The running time of a polynomial-time approximation scheme can increase very rapidly as ϵ decreases.
e.g. $n^{2/\epsilon}$ algorithm

Vertex Cover

Section 34.5.2 defined the vertex-cover problem and proved it NP-complete. Recall that a ***vertex cover*** of an undirected graph $G = (V, E)$ is a subset $V' \subseteq V$ such that if (u, v) is an edge of G , then either $u \in V'$ or $v \in V'$ (or both). The size of a vertex cover is the number of vertices in it.

The ***vertex-cover problem*** is to find a vertex cover of minimum size in a given undirected graph. We call such a vertex cover an ***optimal vertex cover***. This problem is the optimization version of an NP-complete decision problem.

Examples from wiki

- https://en.wikipedia.org/wiki/Vertex_cover

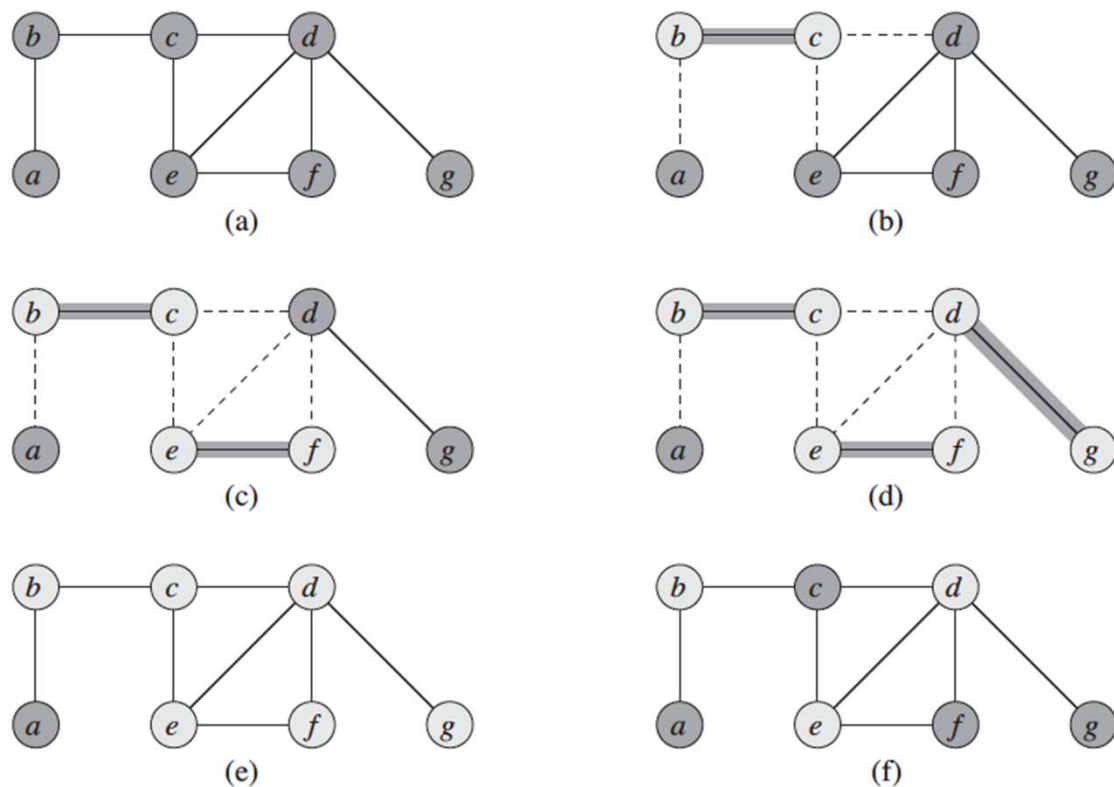


Figure 35.1 The operation of APPROX-VERTEX-COVER. (a) The input graph G , which has 7 vertices and 8 edges. (b) The edge (b, c) , shown heavy, is the first edge chosen by APPROX-VERTEX-COVER. Vertices b and c , shown lightly shaded, are added to the set C containing the vertex cover being created. Edges (a, b) , (c, e) , and (c, d) , shown dashed, are removed since they are now covered by some vertex in C . (c) Edge (e, f) is chosen; vertices e and f are added to C . (d) Edge (d, g) is chosen; vertices d and g are added to C . (e) The set C , which is the vertex cover produced by APPROX-VERTEX-COVER, contains the six vertices b, c, d, e, f, g . (f) The optimal vertex cover for this problem contains only three vertices: b, d , and e .

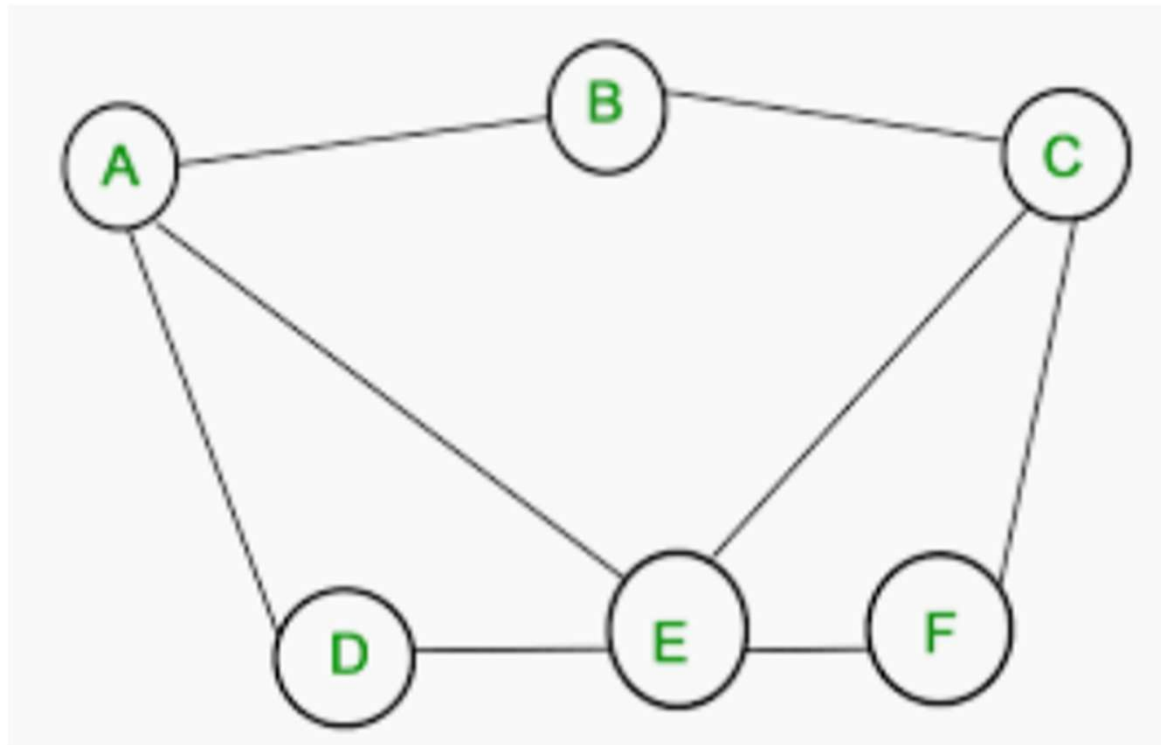
Factor 2 algorithm

APPROX-VERTEX-COVER(G)

```
1   $C = \emptyset$ 
2   $E' = G.E$ 
3  while  $E' \neq \emptyset$ 
4      let  $(u, v)$  be an arbitrary edge of  $E'$ 
5       $C = C \cup \{u, v\}$ 
6      remove from  $E'$  every edge incident on either  $u$  or  $v$ 
7  return  $C$ 
```

- The time complexity is $O(V+E)$ using adjacency list data structure for E'

Find vertex cover



Theorem 35.1

APPROX-VERTEX-COVER is a polynomial-time 2-approximation algorithm.

Proof We have already shown that APPROX-VERTEX-COVER runs in polynomial time.

The set C of vertices that is returned by APPROX-VERTEX-COVER is a vertex cover, since the algorithm loops until every edge in $G.E$ has been covered by some vertex in C .

To see that APPROX-VERTEX-COVER returns a vertex cover that is at most twice the size of an optimal cover, let A denote the set of edges that line 4 of APPROX-VERTEX-COVER picked. In order to cover the edges in A , any vertex cover—in particular, an optimal cover C^* —must include at least one endpoint of each edge in A . No two edges in A share an endpoint, since once an edge is picked in line 4, all other edges that are incident on its endpoints are deleted from E' in line 6. Thus, no two edges in A are covered by the same vertex from C^* , and we have the lower bound

$$|C^*| \geq |A| \tag{35.2}$$

on the size of an optimal vertex cover. Each execution of line 4 picks an edge for which neither of its endpoints is already in C , yielding an upper bound (an exact upper bound, in fact) on the size of the vertex cover returned:

$$|C| = 2|A|. \tag{35.3}$$

Combining equations (35.2) and (35.3), we obtain

$$\begin{aligned} |C| &= 2|A| \\ &\leq 2|C^*|, \end{aligned}$$

thereby proving the theorem. ■

Can we do better?

- For specific version of problem

The approximation factor is $11/6$ for a graph where the maximum degree of the graph is 3.

Alg: Choose the vertex with maximum degree.

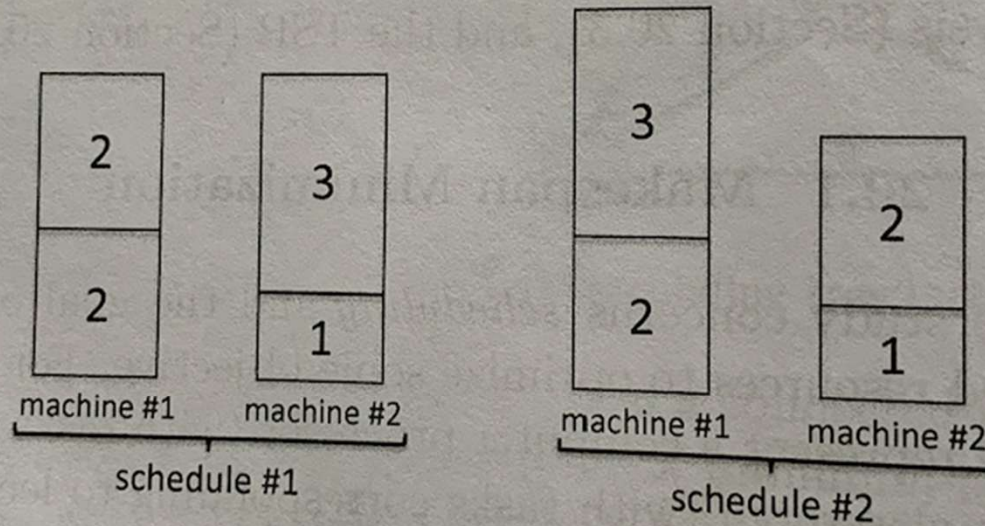
Vertex cover (method 2)

- Linear programming
(https://en.wikipedia.org/wiki/Linear_programming)
- Integer Linear Programming (ILP)
https://en.wikipedia.org/wiki/Integer_programming

Minimum makespan scheduling

- m machines
- n jobs
- Each machine handles a subset of n jobs
- Distribute or schedule jobs among machines
- Minimize completion time for jobs.

What are the makespans of the following schedules? (Jobs are labeled with their lengths.)



- a) 4 and 3
- b) 4 and 4
- c) 4 and 5
- d) 8 and 8

Problem 10.1 (Minimum makespan scheduling) Given processing times for n jobs, $p_1, p_2, \dots, p_n$, and an integer m , find an assignment of the jobs to m identical machines so that the completion time, also called the *makespan*, is minimized.

We will give a simple factor 2 algorithm for this problem before presenting a PTAS for it.

10.1 Factor 2 algorithm

The algorithm is very simple: schedule the jobs one by one, in an arbitrary order, each job being assigned to a machine with least amount of work so far. This algorithm is based on the following two lower bounds on the optimal makespan, OPT:

1. The average time for which a machine has to run, $(\sum_i p_i) / m$; and
2. The largest processing time.

Minimum MakeSpan Scheduling

Algorithm 10.2 (Minimum makespan scheduling)

1. Order the jobs arbitrarily.
2. Schedule jobs on machines in this order, scheduling the next job on the machine that has been assigned the least amount of work so far.

Theorem 10.3 *Algorithm 10.2 achieves an approximation guarantee of 2 for the minimum makespan problem.*

Run time

- Graham's algorithm
- A set of m machines and a set of n jobs
- Using exhaustive search to find the next machine out of m machines for a job
- $T = O(mn)$
- Can we do better?

Run time

- A set of m machines and n **jobs**
- Using exhaustive search to find the next machine out of **m machines** for a job

$$T = O(nm)$$

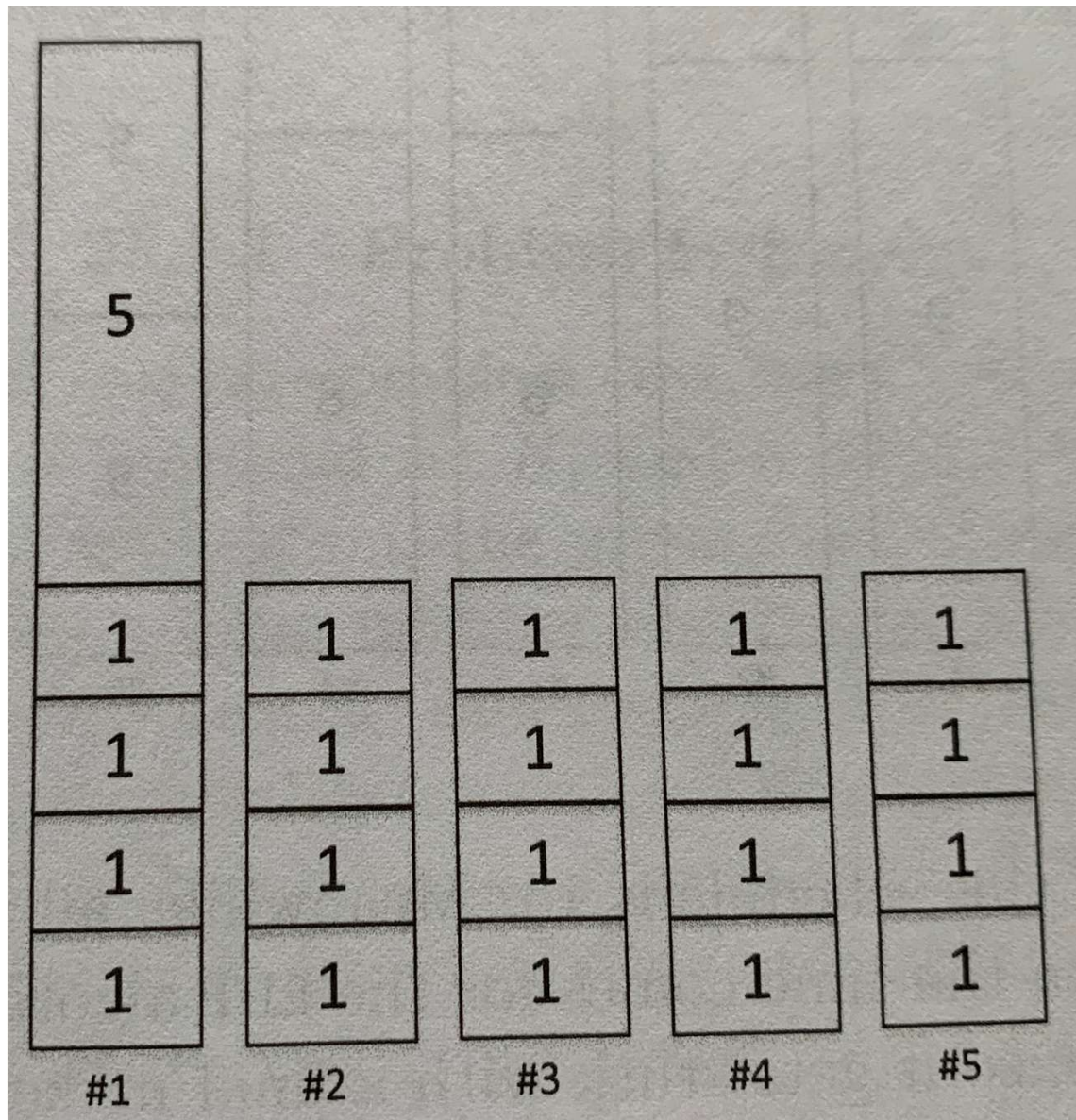
- Can we do better?

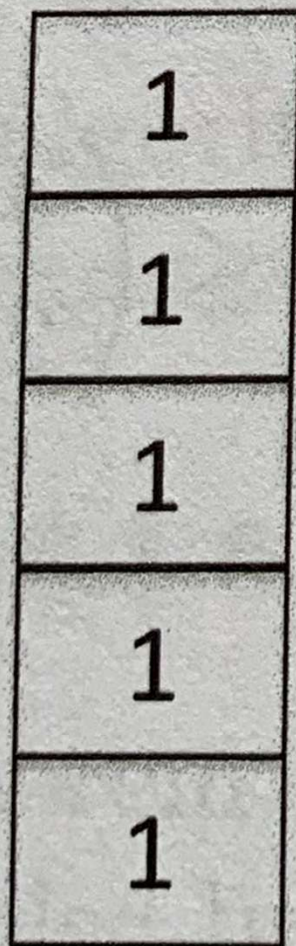
Use min-heap for finding least loaded machine.

$$T = O(n \log m)$$

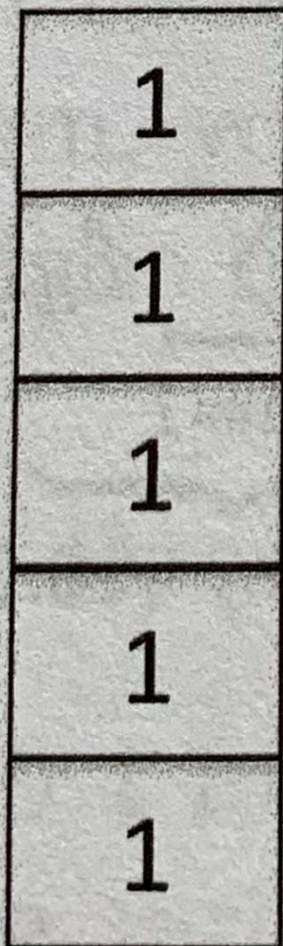
Suppose there are five machines and the list of jobs consists of twenty jobs with length 1 each, followed by a single job with length 5. What is the makespan of the schedule output by the **Graham** algorithm, and what is the smallest-possible makespan of a schedule of these jobs?

- a) 5 and 4
- b) 6 and 5
- c) 9 and 5
- d) 10 and 5

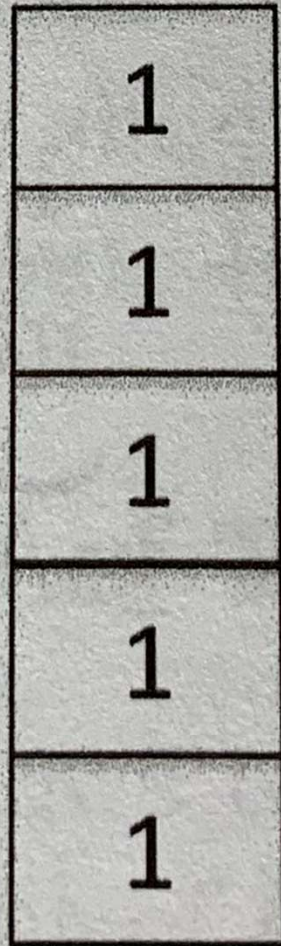




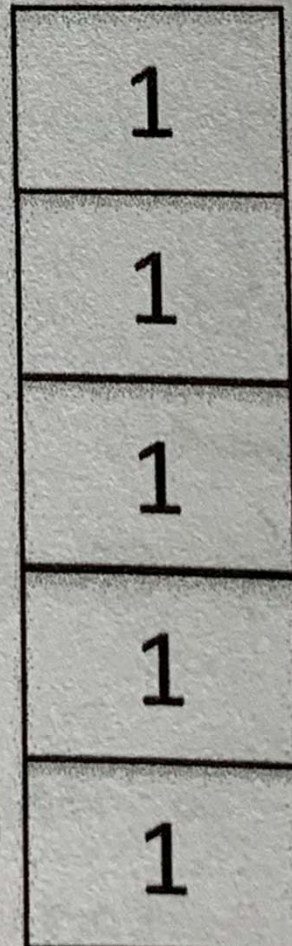
#1



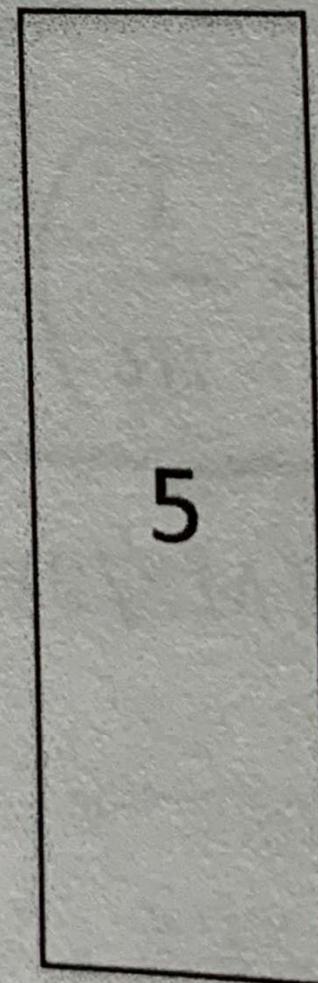
#2



#3



#4



#5

Let LB denote the combined lower bound, i.e.,

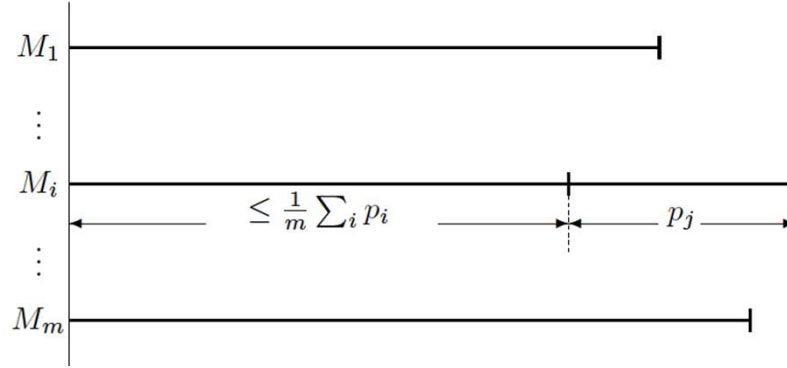
$$\text{LB} = \max \left\{ \frac{1}{m} \sum_i p_i, \max_i \{p_i\} \right\}.$$

Algorithm 10.2 (Minimum makespan scheduling)

1. Order the jobs arbitrarily.
2. Schedule jobs on machines in this order, scheduling the next job on the machine that has been assigned the least amount of work so far.

Theorem 10.3 *Algorithm 10.2 achieves an approximation guarantee of 2 for the minimum makespan problem.*

Proof: Let M_i be the machine that completes its jobs last in the schedule produced by the algorithm, and let j be the index of the last job scheduled on this machine.



Let $start_j$ be the time at which job j starts execution on M_i . Since the algorithm assigns a job to the least loaded machine, it follows that all machines are busy until $start_j$. This implies that

$$start_j \leq \frac{1}{m} \sum_i p_i \leq \text{OPT}.$$

Further, $p_j \leq \text{OPT}$. Thus, the makespan of the schedule is $start_j + p_j \leq 2 \cdot \text{OPT}$. $\square$

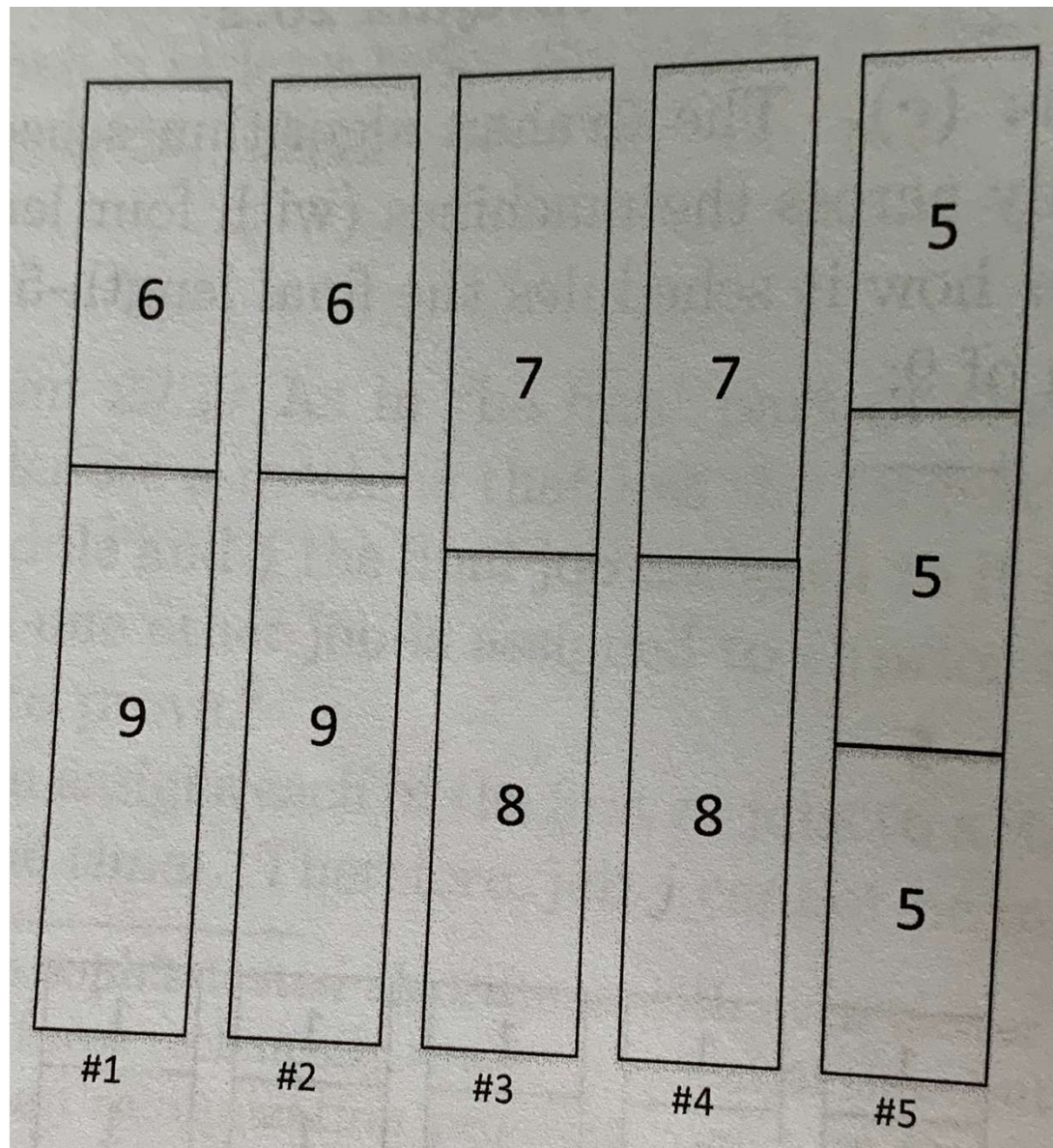
Example 10.4 A tight example for this algorithm is provided by a sequence of m^2 jobs with unit processing time, followed by a single job of length m . The schedule obtained by the algorithm has a makespan of $2m$, while $\text{OPT} = m + 1$. $\square$

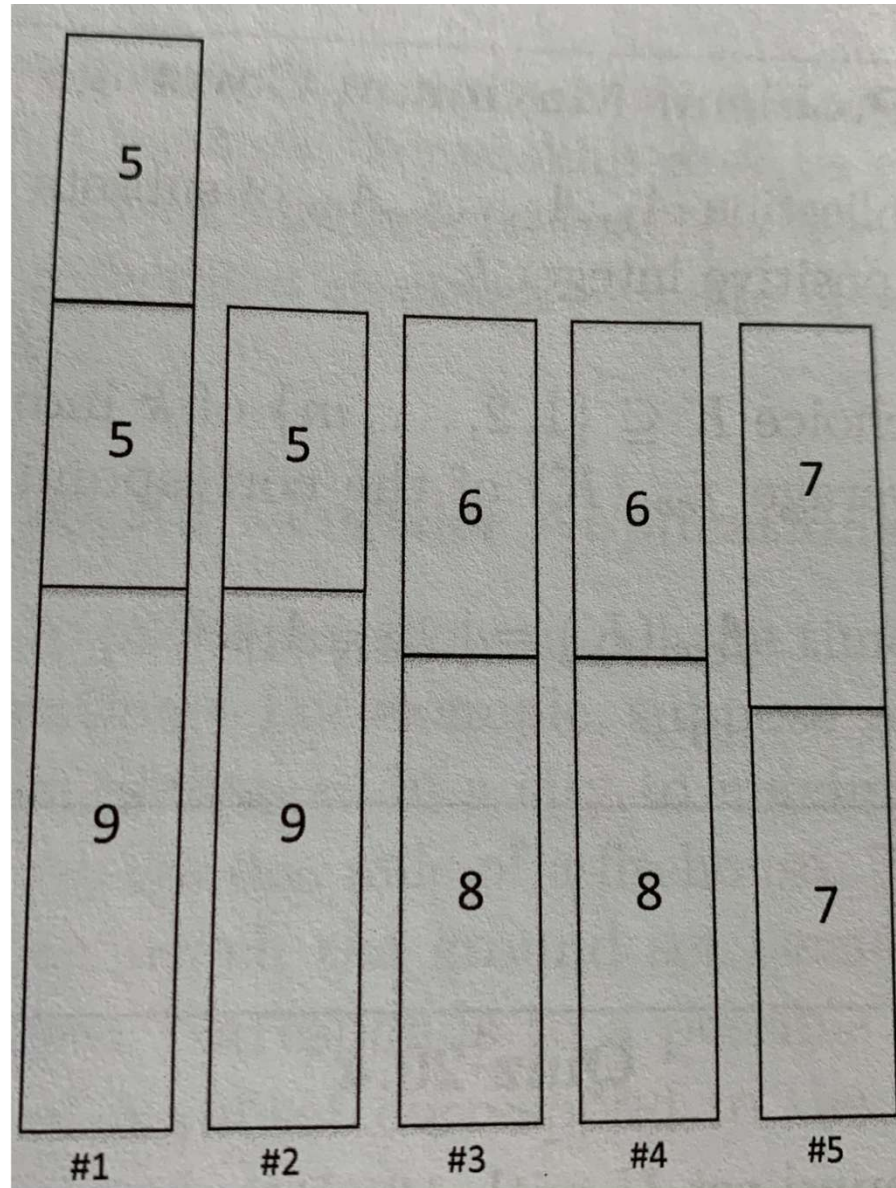
Heuristics: Longest Processing Time First

- Sort the jobs from longest to shortest.
- Run the Graham algorithm on the sorted jobs.

Suppose there are five machines, three jobs with length 5, two with length 6, two with length 7, two with length 8, and two with length 9. What is the makespan of the schedule output by the LPT algorithm, and what is the smallest-possible makespan of a schedule of these jobs?

- a) 16 and 15
- b) 17 and 15
- c) 18 and 15
- d) 19 and 15





LPT Theorem

- The makespan of the schedule output by the LPT algorithm is always at most $\frac{3}{2} - \frac{1}{2m}$ times the minimum-possible makespan, where m denotes the number of machines